ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Thiết kế hệ thống AIoT giám sát môi trường thời gian thực trên ESP32

Internet of Things - CO3038

LỚP: TN01

Giảng viên hướng dẫn: Lê Trọng Nhân
Phan Văn Sỹ

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Lưu Khánh Trình	2313638	TN01	Kỹ thuật máy tính
2	Trần Minh Trí	2313626	TN01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026

Mục lục

1 Giới thiệu **3**
1.1 Tổng quan đề tài . . . 3
1.2 Các nhiệm vụ và yêu cầu của đồ án . . . 3
1.3 Phạm vi và giới hạn . . . 3
1.4 Bố cục báo cáo . . . 4
2 Khái niệm hệ thống và cơ sở lý thuyết **5**
2.1 Nền tảng IoT . . . 5
2.1.1 Thông số kỹ thuật chính . . . 5
2.1.2 Các tính năng liên quan đến đề tài . . . 5
2.2 Cơ sở về RTOS . . . 6
2.2.1 Khái niệm cốt lõi . . . 6
2.2.2 Vai trò của FreeRTOS trong đồ án . . . 6
2.2.3 Kiến trúc tác vụ sử dụng trong đồ án . . . 7
2.3 Cơ sở về TinyML . . . 7
2.3.1 Khái niệm cốt lõi . . . 7
2.3.2 Ưu điểm của TinyML . . . 8
2.3.3 Quy trình phát triển TinyML . . . 8
2.3.4 Ứng dụng TinyML trong đồ án . . . 9
2.3.5 Luồng làm việc trên đám mây và Đường truyền dữ liệu . . . 9
2.3.6 Các thành phần giám sát . . . 9
2.4 Tóm tắt chương . . . 10
3 Thiết kế và triển khai hệ thống **11**
3.1 Tổng quan kiến trúc . . . 11
3.2 Task 1: Nhấp nháy LED theo nhiệt độ . . . 11
3.2.1 Nguyên lý hoạt động . . . 11
3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 12
3.2.3 Sơ đồ máy trạng thái (FSM) . . . 13
3.3 Task 2: Điều khiển NeoPixel theo độ ẩm . . . 13
3.3.1 Nguyên lý hoạt động . . . 13
3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 14
3.3.3 Sơ đồ máy trạng thái (FSM) . . . 14
3.4 Task 3: Giám sát nhiệt độ và độ ẩm với LCD . . . 15
3.4.1 Nguyên lý hoạt động . . . 15
3.4.2 Cơ chế điều phối hệ thống bằng Semaphore . . . 15
3.4.3 Sơ đồ máy trạng thái (FSM) . . . 16
3.5 Task 4: Máy chủ web ở chế độ Access Point . . . 17
3.5.1 Kiến trúc tổng thể . . . 17
3.5.2 Quản lý mạng Wi-Fi và NVS (Non-Volatile Storage) . . . 17
3.5.3 Backend Web Server (API và Routing) . . . 19
3.5.4 Giao diện Frontend (HTML, CSS và JavaScript) . . . 19
3.5.5 Quản lý tác vụ và Luồng xử lý (Task Management) . . . 19
3.6 Task 5: Triển khai TinyML và đánh giá độ chính xác . . . 21
3.6.1 Tổng quan luồng thực thi của TinyML: . . . 21
3.6.2 Thu thập dữ liệu từ cảm biến (Data Collection) . . . 22
3.6.3 Tiền xử lý và Gán nhãn dữ liệu (Data Preprocessing and Labeling) . 24
3.6.4 Đánh giá và So sánh Mô hình (Model Benchmarking) . . . 24

3.6.5 Huấn luyện và Xuất mã C (Model Training and C Header Export) . 27
3.6.6 Kiểm thử mô hình cục bộ (Local Model Testing) 28
3.6.7 Triển khai Firmware trên Vi điều khiển (On-device Deployment) . . 29
3.7 Task 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT 31
3.7.1 Nguyên lý hoạt động 31
3.7.2 Sơ đồ máy trạng thái (FSM) 32
3.8 Tóm tắt chương 32
4 Đánh giá và kết quả thực nghiệm 33
4.1 Kết quả chức năng 33
4.1.1 Task 1 & 2: LED và NeoPixel 33
4.1.2 Task 3: Giám sát môi trường qua LCD 33
4.1.3 Task 4: Máy chủ Web cục bộ (Local Web Server) 34
4.1.4 Task 5: (TinyML) 35
4.1.5 Task 6: Tích hợp đám mây CoreIoT 36
4.2 Đánh giá TinyML 37
4.3 Tóm tắt chương 37
5 Kết luận 38
Tài liệu tham khảo 39

1 Giới thiệu

1.1 Tổng quan đề tài

Đề tài xây dựng hệ thống IoT giám sát môi trường thời gian thực trên vi điều khiển ESP32. Giải pháp tích hợp đồng bộ ba thành phần cốt lõi: nền tảng đa nhiệm thu thập dữ liệu, mạng giao tiếp kép (Web Server nội bộ và đám mây CoreIoT), và mô hình học máy tại biên (TinyML). Qua đó, đề tài nâng cấp vi điều khiển nhúng thành một hệ thống AIoT thông minh, có khả năng tự chủ dự đoán trạng thái môi trường độc lập và tối ưu.

1.2 Các nhiệm vụ và yêu cầu của đồ án

Phần triển khai được tổ chức thành sáu nhiệm vụ chính:

1. **Task 1: Nhấp nháy LED theo nhiệt độ** — điều khiển LED trạng thái nhấp nháy dựa trên các mức nhiệt độ đo được.
2. **Task 2: Điều khiển NeoPixel theo độ ẩm** — thay đổi màu sắc/hành vi của NeoPixel dựa trên các khoảng độ ẩm.
3. **Task 3: Giám sát nhiệt độ và độ ẩm với LCD** — đọc dữ liệu cảm biến liên tục và hiển thị kết quả thời gian thực trên giao diện LCD.
4. **Task 4: Máy chủ web ở chế độ Access Point** — triển khai máy chủ web trên ESP32-S3 để cung cấp chức năng cấu hình và giám sát cục bộ qua chế độ AP.
5. **Task 5: Triển khai TinyML và đánh giá độ chính xác** — đưa mô hình TinyML lên thiết bị và đánh giá chất lượng suy luận bằng các thước đo phù hợp.
6. **Task 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT** — truyền dữ liệu thu thập được và trạng thái thiết bị đến backend CoreIoT để giám sát từ xa.

Toàn bộ hoạt động lập trình, tích hợp và kiểm thử được thực hiện trên nền tảng **ESP32-S3** với **PlatformIO** làm môi trường phát triển chính.

1.3 Phạm vi và giới hạn

Đề Tài tập trung vào việc thiết kế và phát triển phần mềm cho hệ thống IoT đa nhiệm trên nền tảng vi điều khiển ESP32. Hệ thống bao quát các tính năng cốt lõi: xử lý đa tác vụ với hệ điều hành FreeRTOS, giám sát và cấu hình nội bộ qua Máy chủ Web (AP/STA mode), giao tiếp dữ liệu đám mây qua giao thức MQTT (CoreIoT), và triển khai mô hình phân loại tĩnh tại biên (TinyML - Naive Bayes).

1.4 Bố cục báo cáo

Dựa trên quy trình phát triển thực tế, báo cáo được cấu trúc thành 5 chương chính:

- **Chương 1: Giới thiệu :** Nêu bật bối cảnh, lý do chọn đề tài, mục tiêu và xác định rõ phạm vi giới hạn của đồ án.
- **Chương 2: Cơ sở lý thuyết :** Cung cấp các kiến thức nền tảng về vi điều khiển ESP32, hệ điều hành thời gian thực FreeRTOS, giao thức MQTT và kỹ thuật học máy tại biên (TinyML).
- **Chương 3: Thiết kế và triển khai :** Đi sâu vào kiến trúc phần mềm, cơ chế đồng bộ đa nhiệm và mã nguồn của các tác vụ thu thập, cảnh báo, máy chủ Web và giao tiếp đám mây.
- **Chương 4: Đánh giá và thực nghiệm :** Đánh giá tính ổn định của hệ thống, thời gian phản hồi và độ chính xác của mô hình học máy.
- **Chương 5: Kết luận :** Tổng kết những kết quả kỹ thuật đạt được và vạch ra định hướng nâng cấp hệ thống trong tương lai.

2 Khái niệm hệ thống và cơ sở lý thuyết

2.1 Nền tảng IoT

ESP32 là họ vi điều khiển 32-bit do Espressif Systems phát triển, kế thừa từ ESP8266. Nền tảng này tích hợp kết nối Wi-Fi và Bluetooth/BLE, đồng thời cung cấp hiệu năng xử lý đủ tốt cho các ứng dụng IoT biên. Trong đồ án này, ESP32-S3 được dùng làm bộ điều khiển trung tâm cho các tác vụ cảm biến, xử lý cục bộ, truyền thông và tương tác với đám mây.

2.1.1 Thông số kỹ thuật chính

- **Vi xử lý (Processor):** Kiến trúc 32-bit Xtensa/RISC-V (tùy thuộc vào model), thường hoạt động trong dải tần số 160–240 MHz.
- **Bộ nhớ (Memory):** Tài nguyên SRAM tích hợp trên chip với hỗ trợ bộ nhớ Flash ngoài (thông dụng từ 4–16 MB, tùy thuộc vào cấu hình của từng bo mạch).
- **Kết nối (Connectivity):** Hỗ trợ Wi-Fi chuẩn IEEE 802.11 b/g/n và Bluetooth Low Energy (BLE).
- **Giao tiếp ngoại vi (Peripheral Interfaces):** Tích hợp các chuẩn giao tiếp đa dạng như GPIO, PWM, ADC, UART, SPI và I^2C.
- **Tính năng quản lý năng lượng (Power Features):** Hỗ trợ các chế độ tiết kiệm năng lượng (low-power modes), phù hợp cho các hệ thống nhúng và thiết bị IoT chạy bằng pin.
- **Điều kiện hoạt động (Operating Range):** Đáp ứng các tiêu chuẩn hoạt động trong môi trường công nghiệp cho các kịch bản triển khai thực tế.

2.1.2 Các tính năng liên quan đến đề tài

Trong đồ án này, các khả năng sau của ESP32 được sử dụng trực tiếp:

- **Giao tiếp I^2C:** Thực hiện kết nối với các cảm biến kỹ thuật số để thu thập dữ liệu nhiệt độ và độ ẩm.
- **GPIO/PWM:** Điều khiển trạng thái đèn LED, cấu hình hành vi của NeoPixel và các cơ cấu chấp hành khác.
- **Hỗ trợ máy chủ Web (Web server support):** Cho phép giám sát và điều khiển thiết bị cục bộ thông qua chế độ Access Point (AP).
- **Hỗ trợ hệ thống tệp trên Flash (ví dụ: LittleFS):** Lưu trữ các tài nguyên web tĩnh (HTML/CSS/JS).

- **Tương thích TinyML:** Khả năng suy luận trực tiếp trên thiết bị để phân tích trạng thái môi trường một cách nhẹ nhàng và tức thời.

2.2 Cơ sở về RTOS

FreeRTOS là một nhân hệ điều hành thời gian thực (RTOS) gọn nhẹ, mã nguồn mở, được thiết kế cho các hệ thống nhúng có tài nguyên hạn chế. Hệ điều hành này cung cấp một lớp trừu tượng có cấu trúc cho thực thi đồng thời, xử lý sự kiện và đồng bộ giữa các tác vụ mà không yêu cầu lập trình viên phải tự xây dựng cơ chế lập lịch phức tạp. Nhờ chi phí tài nguyên thấp và hệ sinh thái hỗ trợ trưởng thành, FreeRTOS được sử dụng rộng rãi trên các nền tảng vi điều khiển như ESP32, STM32 và ARM Cortex-M.

2.2.1 Khái niệm cốt lõi

- **Task (Tác vụ):** là một đơn vị thực thi độc lập, thường được triển khai dưới dạng một vòng lặp vô hạn với không gian bộ nhớ (stack) và trạng thái thực thi riêng.
- **Scheduler (Bộ lập lịch):** thành phần cốt lõi quyết định tác vụ nào được thực thi tại mỗi thời điểm; FreeRTOS thường sử dụng cơ chế lập lịch ưu tiên trưng dụng (preemptive priority-based scheduling).
- **Queue (Hàng đợi):** cơ chế truyền thông an toàn luồng (thread-safe) để truyền tin nhắn hoặc dữ liệu giữa các tác vụ.
- **Semaphore:** một cơ chế đồng bộ hóa được sử dụng để báo hiệu các sự kiện và kiểm soát quyền truy cập vào các tài nguyên dùng chung.
- **Mutex:** một cơ chế khóa chuyên dụng để bảo vệ các tài nguyên dùng chung quan trọng khỏi việc bị truy cập đồng thời.

2.2.2 Vai trò của FreeRTOS trong đồ án

Trong hệ thống IoT này, FreeRTOS được dùng để điều phối nhiều mô-đun có yêu cầu khắt khe về mặt thời gian và truyền thông:

1. **Quản lý tác vụ đồng thời:** các tác vụ riêng biệt đảm nhiệm việc xử lý web server, lấy mẫu cảm biến định kỳ, thực thi suy luận TinyML và điều khiển LED/thiết bị.
2. **Đồng bộ hóa dữ liệu:** semaphore bảo vệ các luồng dữ liệu cảm biến dùng chung, trong khi hàng đợi hỗ trợ việc truyền dữ liệu an toàn giữa các tác vụ.
3. **Phản hồi theo mức độ ưu tiên:** tác vụ xử lý yêu cầu web có thể được gán mức ưu tiên cao hơn để đảm bảo khả năng tương tác mượt mà với người dùng; các tác vụ thu thập cảm biến và suy luận được chạy ở mức ưu tiên trung bình một cách có kiểm soát.

4. **Cô lập tài nguyên:** mỗi tác vụ có một vùng nhớ (stack) chuyên dụng riêng, giúp giảm thiểu hiện tượng tràn/nhiễu bộ nhớ và cải thiện độ ổn định của toàn hệ thống.

2.2.3 Kiến trúc tác vụ sử dụng trong đồ án

Thiết kế phần mềm tuân theo mô hình tổ chức đa tác vụ của FreeRTOS:

- **Task WebServer (mức ưu tiên cao):** xử lý các yêu cầu HTTP và các tương tác trên giao diện người dùng cục bộ.
- **Task Sensor Reader (mức ưu tiên trung bình):** thu thập dữ liệu đo nhiệt độ và độ ẩm định kỳ.
- **Task TinyML (mức ưu tiên trung bình):** thực thi quá trình suy luận trực tiếp trên thiết bị (on-device inference) để dự đoán trạng thái.
- **Task Điều khiển LED/Thiết bị:** cập nhật trạng thái đầu ra của LED/NeoPixel và các cơ cấu chấp hành liên quan.
- **Task Quản lý Wi-Fi:** quản lý kết nối mạng và logic tự động kết nối lại khi mất tín hiệu.
- **Task Nhàn rỗi - Idle Task (mức ưu tiên thấp nhất):** chạy khi không có tác vụ ứng dụng nào sẵn sàng thực thi.

2.3 Cơ sở về TinyML

TinyML (Tiny Machine Learning) tập trung vào việc triển khai suy luận học máy trên các thiết bị nhúng có tài nguyên hạn chế như vi điều khiển và cảm biến thông minh. Thay vì chuyển luồng dữ liệu cảm biến thô lên máy chủ đám mây để xử lý AI, TinyML thực hiện dự đoán trực tiếp ở biên. Cách tiếp cận này giúp giảm độ trễ, giảm chi phí năng lượng cho truyền thông, tăng cường riêng tư và cho phép hệ thống tiếp tục hoạt động ngay cả khi mạng không ổn định.

2.3.1 Khái niệm cốt lõi

- **Suy luận (Inference):** Sử dụng mô hình đã được huấn luyện trước để dự đoán các mẫu dữ liệu đầu vào mới; TinyML chủ yếu tập trung vào quá trình suy luận thay vì huấn luyện trực tiếp trên thiết bị (on-device training).
- **Lượng tử hóa mô hình (Model quantization):** Giảm độ chính xác của các trọng số trong mô hình (ví dụ: từ float32 xuống int8 hoặc int16) nhằm giảm dung lượng bộ nhớ và cải thiện hiệu suất thực thi.

- **TensorFlow Lite / TensorFlow Lite Micro:** Các môi trường thực thi (runtimes) tối giản, được thiết kế để triển khai các mô hình nhỏ gọn trên phần cứng nhúng và thiết bị IoT.
- **Trích xuất đặc trưng (Feature extraction):** Chuyển đổi dữ liệu cảm biến thô thành các đặc trưng có ý nghĩa, phù hợp làm đầu vào cho mô hình.
- **Trí tuệ nhân tạo biên (Edge AI):** Thực hiện các tính toán AI ngay tại nguồn tạo ra dữ liệu (thiết bị biên) thay vì phụ thuộc vào cơ sở hạ tầng đám mây từ xa.

2.3.2 Ưu điểm của TinyML

Trong đồ án này, TinyML mang lại một số lợi ích thực tiễn:

1. **Độ trễ thấp:** Các dự đoán được thực hiện cục bộ với thời gian gần như thực (near real-time).
2. **Tiết kiệm năng lượng:** Việc giảm thiểu truyền tải dữ liệu qua mạng giúp giảm đáng kể mức tiêu thụ điện năng.
3. **Bảo mật dữ liệu:** Các dữ liệu đo lường thô và nhạy cảm có thể được giữ lại và xử lý hoàn toàn trên thiết bị.
4. **Khả năng hoạt động ngoại tuyến (Offline):** Quá trình suy luận cốt lõi vẫn tiếp tục diễn ra ngay cả khi không có kết nối với máy chủ đám mây.
5. **Triển khai nhỏ gọn:** Các mô hình sau khi tối ưu hóa có thể hoạt động trơn tru trong giới hạn khắt khe về bộ nhớ và khả năng tính toán.
6. **Giảm chi phí hạ tầng:** Ít phụ thuộc hơn vào các dịch vụ tính toán đám mây có dung lượng lớn và chi phí cao.

2.3.3 Quy trình phát triển TinyML

Quy trình triển khai thường bao gồm các bước sau:

1. Huấn luyện mô hình ngoại tuyến bằng cách sử dụng các framework học máy trên Python và tập dữ liệu lịch sử.
2. Chuyển đổi mô hình đã huấn luyện sang định dạng triển khai nhẹ (ví dụ: `.tflite` hoặc các định dạng biểu diễn nhỏ gọn tương đương).
3. Áp dụng các kỹ thuật tối ưu hóa và lượng tử hóa để giảm mức sử dụng bộ nhớ và tăng tốc độ thực thi.

4. Tích hợp các tham số của mô hình vào firmware (ví dụ: dưới dạng tệp tin header của ngôn ngữ C).
5. Triển khai lên vi điều khiển ESP32-S3 và đánh giá độ trễ thực thi, mức tiêu thụ bộ nhớ cùng chất lượng dự đoán.

2.3.4 Ứng dụng TinyML trong đồ án

Trong hệ thống IoT này, TinyML được sử dụng để phân loại trạng thái môi trường từ dữ liệu cảm biến thời gian thực:

- **Đặc trưng đầu vào (Input features):** Nhiệt độ (°C) và độ ẩm (%) thu thập từ cảm biến DHT20.
- **Lớp đầu ra (Output classes):** Các trạng thái hoạt động của môi trường (ví dụ: trạng thái không hợp lệ, chế độ chờ, cần làm mát, cần hút ẩm, cần tạo ẩm).
- **Lựa chọn mô hình:** Thuật toán Naive Bayes với phân phối chuẩn (Gaussian Naive Bayes - GNB), được huấn luyện trên các cặp mẫu nhiệt độ - độ ẩm lịch sử đã được gán nhãn.
- **Tích hợp Firmware:** Các tham số của mô hình được xuất ra tệp `naive_bayes_model.h` và biên dịch cùng với ứng dụng trên ESP32-S3.

Để giảm thiểu băng thông giao tiếp, hệ thống có thể chỉ gửi đi các kết quả suy luận ngắn gọn (nhãn trạng thái) thay vì liên tục truyền toàn bộ dữ liệu thô.

2.3.5 Luồng làm việc trên đám mây và Đường truyền dữ liệu

CoreIoT là nền tảng quản lý IoT thời gian thực. Hệ thống sử dụng MQTT - giao thức truyền thông Publish/Subscribe hạng nhẹ, đặc biệt tối ưu cho vi điều khiển ESP32. Luồng dữ liệu hai chiều được định tuyến như sau:

- **Chiều lên (Telemetry):** Trạng thái môi trường và thông số ngưỡng được đóng gói thành định dạng JSON, xuất bản (Publish) định kỳ lên đám mây tại topic `v1/devices/me/telemetry`.
- **Chiều xuống (RPC Control):** Đám mây đóng vai trò Broker điều hướng lệnh điều khiển. ESP32 đăng ký lắng nghe (Subscribe) tại topic `v1/devices/me/rpc/request/+` để bắt sự kiện từ người dùng và phản hồi xuống phần cứng.

2.3.6 Các thành phần giám sát

Không gian điều khiển của hệ thống được tích hợp trên Dashboard thông qua hai nhóm thành phần chính:

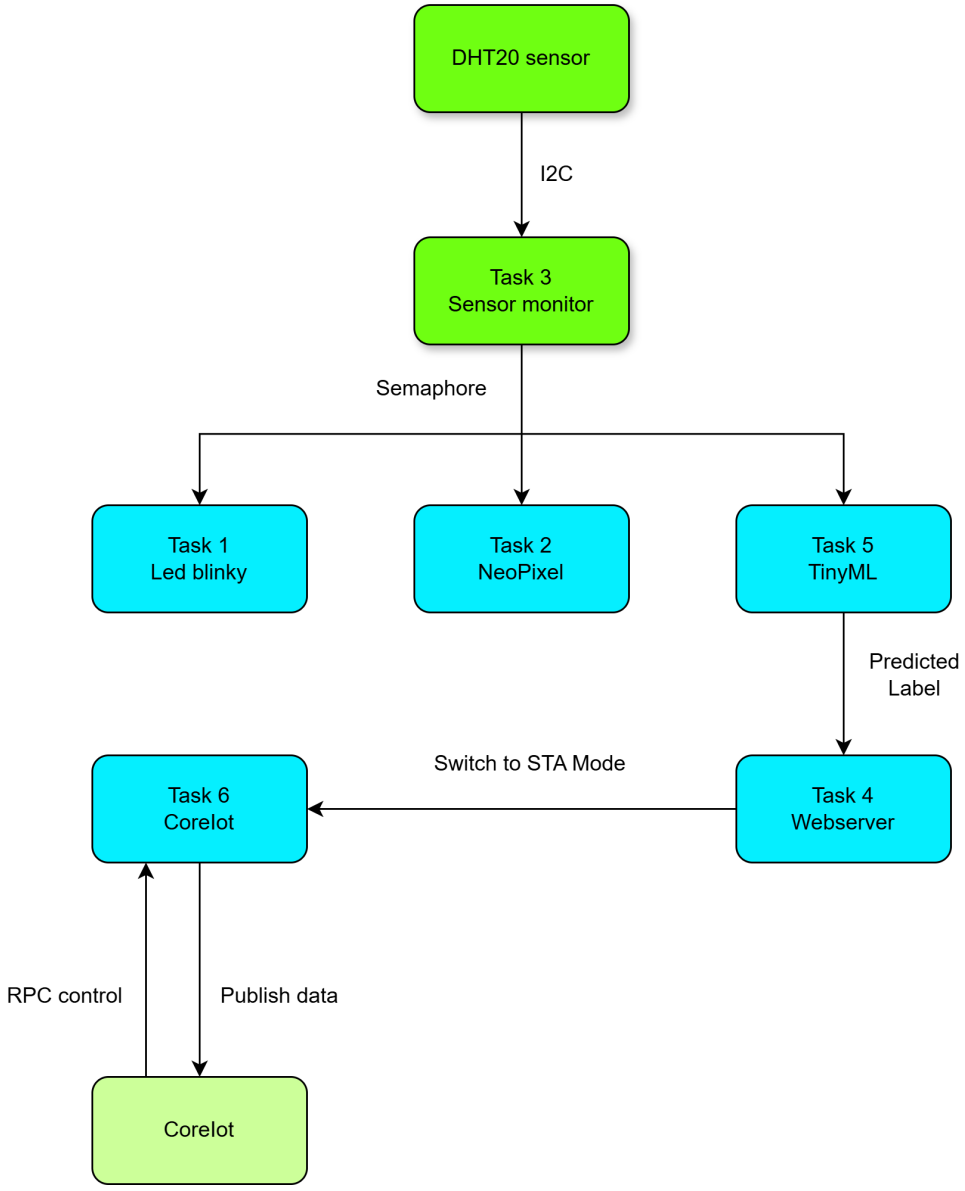
- **Thành phần hiển thị:** Gồm Thẻ số (Digital Cards) và Biểu đồ đường (Line Charts) để giám sát và lưu trữ chuỗi dữ liệu (time-series) của cảm biến.
- **Thành phần điều khiển:** Gồm thanh trượt (Slider) và công tắc (Switch) liên kết với các phương thức RPC (`setThre`, `setStateLED`). Điều này cho phép thay đổi logic điều khiển trực tiếp trên nền tảng Web.

2.4 Tóm tắt chương

Chương này đã trình bày quá trình thiết kế và triển khai phần mềm cho hệ thống IoT. Nhờ ứng dụng hệ điều hành FreeRTOS, ESP32 đã vận hành trên kiến trúc đa nhiệm gồm: module thu thập dữ liệu (DHT20, LCD), cảnh báo trực quan (LED, NeoPixel), Web Server thiết lập mạng và giao tiếp đám mây CoreIoT qua MQTT.

3 Thiết kế và triển khai hệ thống

3.1 Tổng quan kiến trúc



Hình 1: Kiến trúc hệ thống

3.2 Task 1: Nhấp nháy LED theo nhiệt độ

3.2.1 Nguyên lý hoạt động

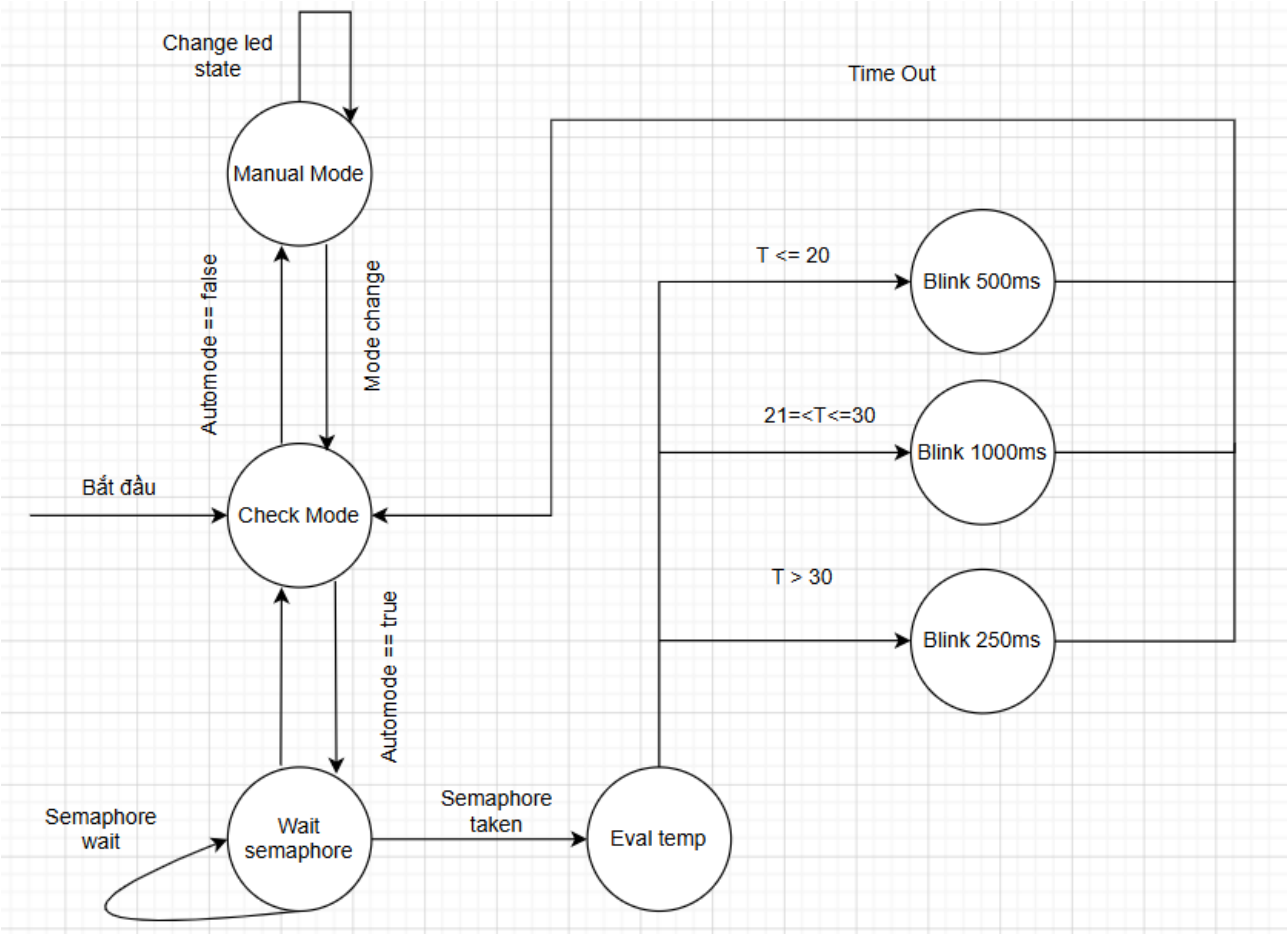
Tác vụ "Temperature-Driven LED Blinking"được thiết kế nhằm điều khiển trạng thái chớp tắt của đèn cảnh báo LED1 thông qua dữ liệu nhiệt độ môi trường hoặc sự can thiệp trực tiếp từ người dùng. Hệ thống hoạt động dựa trên cơ chế chuyển đổi linh hoạt giữa hai trạng thái chính: *Auto Mode* (Chế độ tự động) và *Manual Mode* (Chế độ thủ công).

- **Manual Mode (Chế độ thủ công):** Khi cờ `led_auto_mode` được đặt thành `false`, hệ thống sẽ bỏ qua các dữ liệu từ cảm biến. Trạng thái của LED1 lúc này được quyết định hoàn toàn bởi biến `led_manual_1`, cho phép phản hồi tức thời với các tín hiệu điều khiển từ xa qua Web Server hoặc nền tảng CoreIoT.
- **Auto Mode (Chế độ tự động):** Khi `led_auto_mode` là `true`, tác vụ sẽ chờ tín hiệu đồng bộ (Semaphore) từ quá trình đọc cảm biến. Tần số chớp tắt của LED được chia thành 3 dải nhiệt (T) như sau:
  - $T \leq 20°C$: Môi trường lạnh, chu kỳ nhấp nháy là 500ms.
  - $21°C \leq T \leq 30°C$: Môi trường ổn định, chu kỳ nhấp nháy chậm ở mức 1000ms.
  - $T > 30°C$: Cảnh báo nhiệt độ cao, chu kỳ nhấp nháy nhanh ở mức 250ms để thu hút sự chú ý.

3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Được triển khai dưới dạng một FreeRTOS Task độc lập (`led_blinky`), hệ thống tránh được hiện tượng thắt cổ chai (blocking) khi xử lý các hàm `vTaskDelay`. Nhằm đảm bảo tính toàn vẹn dữ liệu (Thread-safe) giữa Task đọc cảm biến và Task điều khiển LED, một Binary Semaphore (`xBinarySemaphoreTemp_blinky`) đã được áp dụng. Task LED chỉ lấy giá trị nhiệt độ mới vào biến cục bộ `local_temp` khi chiếm được quyền kiểm soát Semaphore, qua đó ngăn chặn tuyệt đối lỗi "Data Race".

3.2.3 Sơ đồ máy trạng thái (FSM)



Hình 2: Task 1 FSM

3.3 Task 2: Điều khiển NeoPixel theo độ ẩm

3.3.1 Nguyên lý hoạt động

Tác vụ "Humidity and Network-Driven NeoPixel Control"chịu trách nhiệm điều khiển đèn LED RGB (NeoPixel) để phản ánh trực quan tình trạng độ ẩm không khí và trạng thái kết nối mạng của hệ thống. Tương tự như LED1, dải NeoPixel cũng hỗ trợ hai chế độ hoạt động: *Auto Mode* và *Manual Mode*.

- **Manual Mode (Chế độ thủ công):** Khi `led_auto_mode` bị tắt, màu sắc của NeoPixel được xác định bởi các giá trị `led_manual_r`, `led_manual_g`, `led_manual_b` và được tinh chỉnh độ sáng thông qua biến `led_brightness` (tính theo phần trăm). Sau khi cập nhật màu sắc, vòng lặp sẽ tạm nghỉ 150ms.
- **Auto Mode (Chế độ tự động):** Ở chế độ này, hệ thống sẽ ưu tiên kiểm tra trạng thái kết nối mạng, sau đó mới đánh giá dữ liệu độ ẩm (H) từ cảm biến. Các mức hiển thị màu sắc được quy định cụ thể như sau:

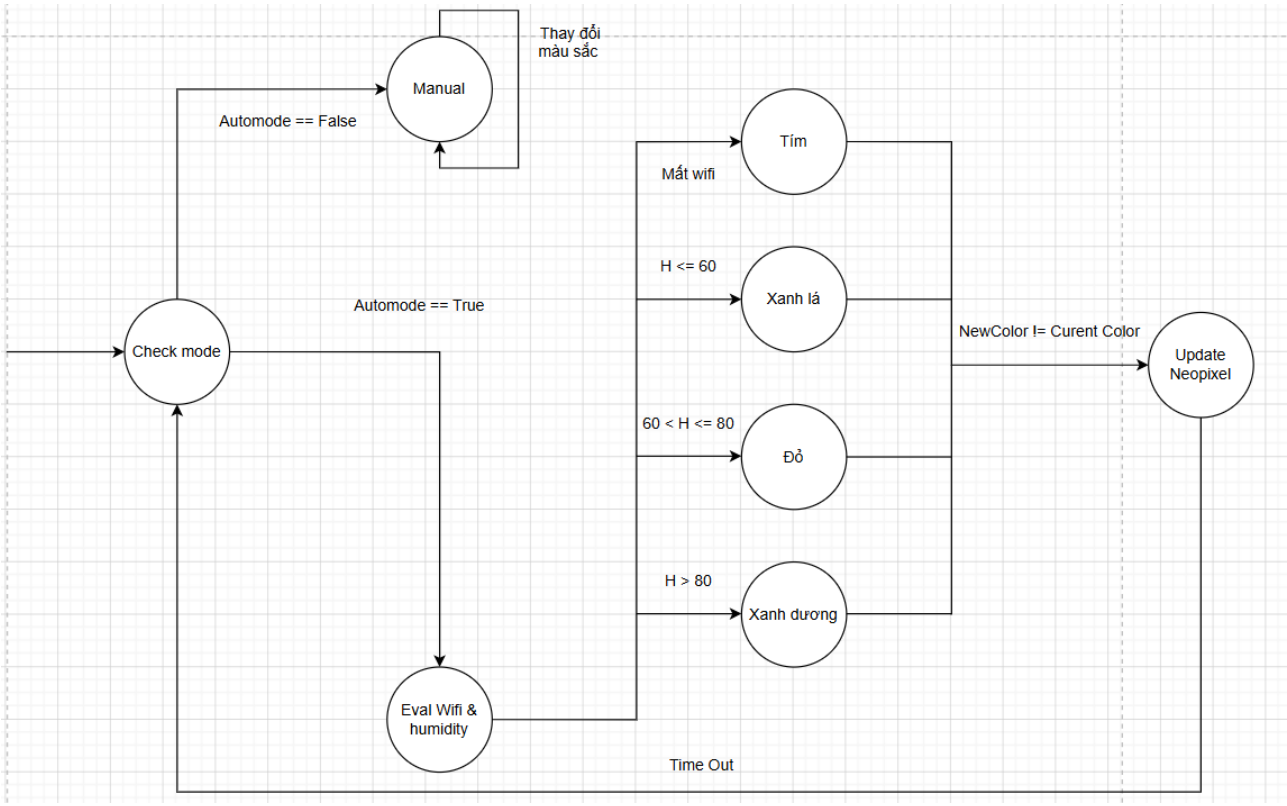
- **Mất kết nối Wi-Fi** (`!isWifiConnected`): Đèn hiển thị màu Tím (Purple) cảnh báo mất mạng độc lập với dữ liệu cảm biến.
- $H \leq 60\%$: Độ ẩm lý tưởng, đèn hiển thị màu Xanh lá (Green).
- $60\% < H \leq 80\%$: Độ ẩm hơi cao, đèn hiển thị màu Đỏ (Red).
- $H > 80\%$: Độ ẩm bão hòa, đèn hiển thị màu Xanh dương (Blue).

Tác vụ chạy với chu kỳ delay là 300ms để đảm bảo cập nhật trạng thái liên tục mà không tiêu tốn tài nguyên hệ thống.

3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Tác vụ `neo_blinky` sử dụng hàm `xSemaphoreTake` để kiểm tra việc có hay không dữ liệu mới từ cảm biến. Khi nhận được Semaphore từ Task đọc cảm biến, biến cục bộ `local_hum` sẽ được cập nhật bằng giá trị mới nhất từ biến toàn cục `glob_humidity`. Điều này giúp vòng lặp của NeoPixel không bị chặn lại nếu cảm biến chưa đọc xong, duy trì khả năng cảnh báo mất Wi-Fi tức thời. Hệ thống cũng áp dụng kỹ thuật so sánh cờ `currentColor` và `newColor` để chỉ gọi hàm `strip.show()` khi thực sự có sự thay đổi màu sắc,

3.3.3 Sơ đồ máy trạng thái (FSM)



Hình 3: Task 2 FSM

3.4 Task 3: Giám sát nhiệt độ và độ ẩm với LCD

3.4.1 Nguyên lý hoạt động

Tác vụ "Temperature and Humidity Monitoring"đóng vai trò là khối thu thập dữ liệu trung tâm (Producer) của toàn bộ hệ thống. Tác vụ này có trách nhiệm giao tiếp với cảm biến DHT20 qua chuẩn I2C để trích xuất nhiệt độ và độ ẩm, hiển thị trực quan lên màn hình LCD 16x2, đồng thời cung cấp dữ liệu đầu vào cho các tác vụ phân tích và cảnh báo khác.

Nguyên lý hoạt động được chia thành các bước kiểm duyệt và xử lý chặt chẽ:

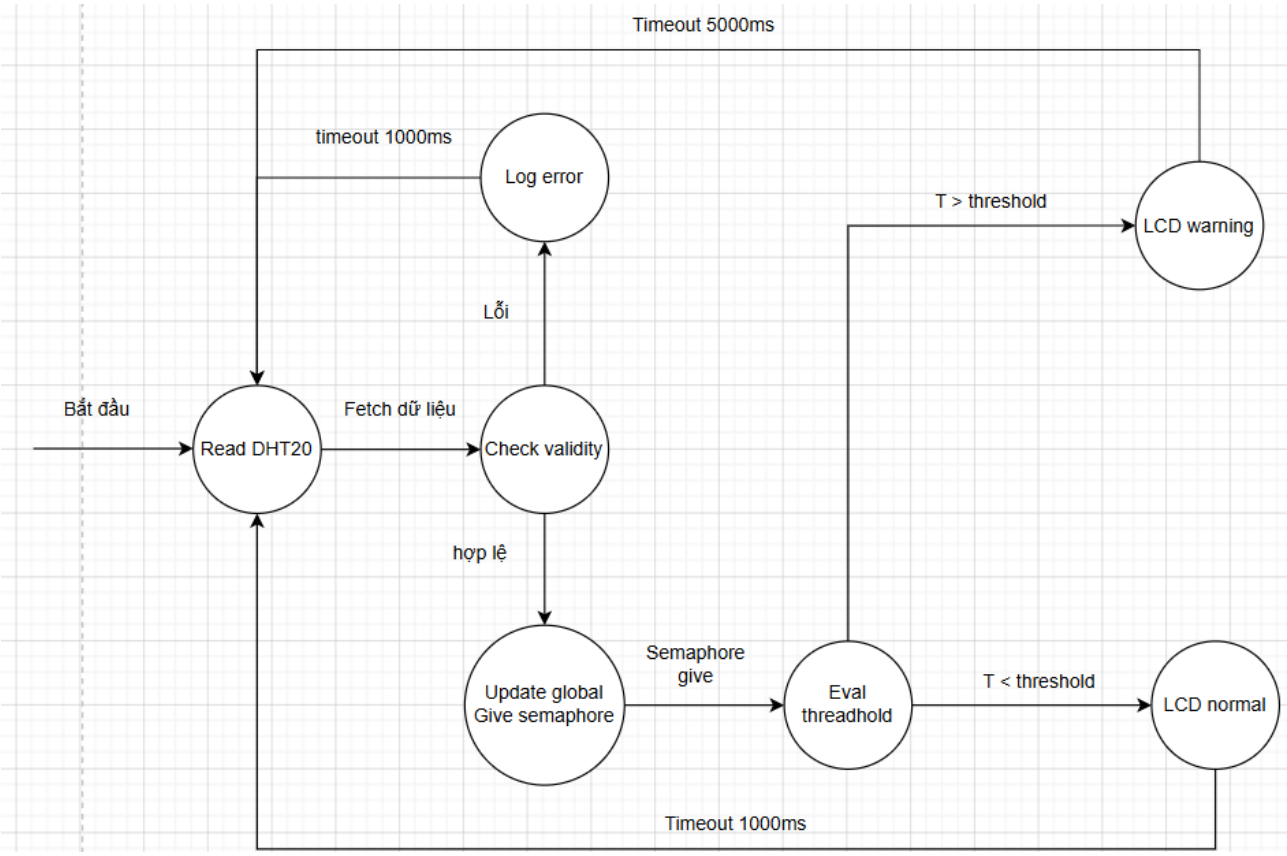
- **Kiểm duyệt dữ liệu (Data Validation):** Sau mỗi lần đọc cảm biến, hệ thống sử dụng hàm `isnan()` để kiểm tra tính hợp lệ của dữ liệu. Nếu xuất hiện lỗi đọc (NaN), hệ thống sẽ in cảnh báo ra Serial, tạm dừng 1000ms và lặp lại quá trình đọc mà không làm thay đổi trạng thái của các biến toàn cục. Điều này giúp bảo vệ hệ thống khỏi các giá trị nhiễu hoặc rác.
- **Hiển thị có điều kiện (Conditional Display):** Dữ liệu hợp lệ sẽ được so sánh với ngưỡng nhiệt độ `glob_temp_threshold` (có thể được cập nhật động từ nền tảng CoreIoT).
  - **Môi trường bình thường ($T \leq$ Threshold):** LCD hiển thị trực quan Độ ẩm (H) ở hàng trên và Nhiệt độ (T) ở hàng dưới. Hệ thống duy trì trạng thái này trong chu kỳ 1000ms trước khi cập nhật mẫu mới.
  - **Vượt ngưỡng cảnh báo ($T >$ Threshold):** LCD ngay lập tức chuyển sang chế độ báo động. Hàng dưới chốt cứng thông số nhiệt độ hiện tại kèm mức ngưỡng vi phạm. Hàng trên thực hiện hiệu ứng chớp tắt dòng chữ `"! WARNING TEMP !"`5 lần (chu kỳ 500ms sáng / 500ms tắt, tổng thời gian 5000ms) để thu hút tối đa sự chú ý của người dùng mà không làm gián đoạn hệ điều hành (nhờ `vTaskDelay`).

3.4.2 Cơ chế điều phối hệ thống bằng Semaphore

Khác với Task 1 và Task 2 đóng vai trò là "Consumer", Task giám sát nhiệt ẩm hoạt động như một "Producer". Ngay khi dữ liệu hợp lệ được ghi vào biến toàn cục `glob_temperature` và `glob_humidity`, tác vụ này lập tức phát tín hiệu đánh thức các phân hệ khác bằng cách giải phóng đồng thời 3 Binary Semaphore:

1. `xBinarySemaphoreTemp_blinky`: Kích hoạt tính toán lại tần số nháy LED1.
2. `xBinarySemaphoreTemp_neo`: Kích hoạt đánh giá màu sắc dải LED NeoPixel.
3. `xBinarySemaphoreTinyMLData`: Kích hoạt bộ nội suy Machine Learning (Naive Bayes) để dự đoán nhãn trạng thái môi trường.

3.4.3 Sơ đồ máy trạng thái (FSM)



Hình 4: Task 3 FSM

3.5 Task 4: Máy chủ web ở chế độ Access Point

3.5.1 Kiến trúc tổng thể

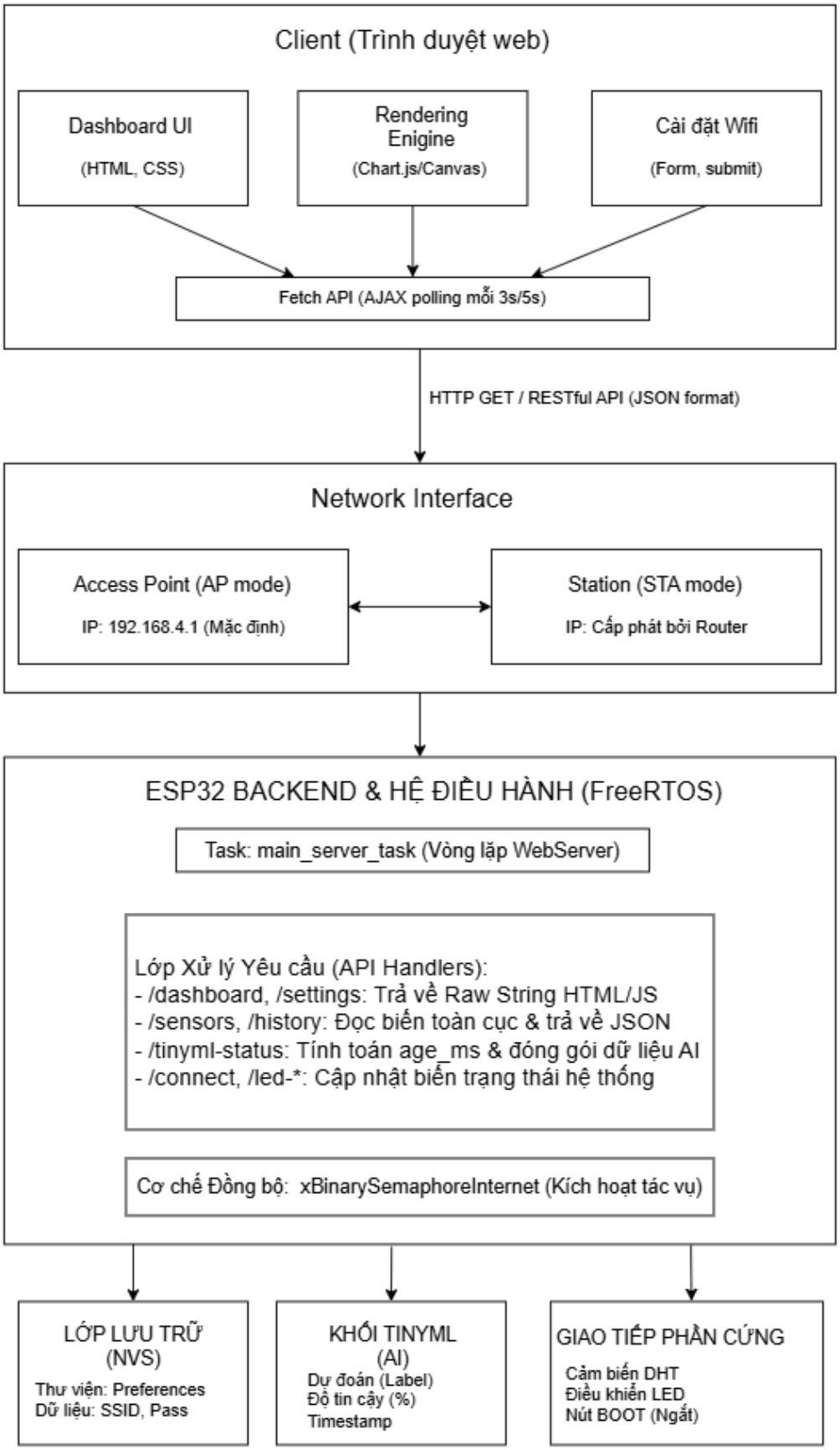
Hệ thống được thiết kế theo mô hình **Client-Server đa luồng (Multi-tasking)** kết hợp với mô hình **Single Page Application (SPA)** ở phía giao diện:

- **Backend (ESP32):** Đóng vai trò là Web Server (cung cấp các API và giao diện HTML) và Router (quản lý kết nối mạng Wi-Fi ở cả chế độ AP và STA).
- **Frontend (Trình duyệt Web):** Giao diện được nhúng trực tiếp vào bộ nhớ Flash của vi điều khiển ESP32. Khi trình duyệt tải trang về, hệ thống sử dụng JavaScript (thông qua Fetch API) để liên tục gọi các API của Backend, từ đó cập nhật dữ liệu cảm biến và trạng thái mới nhất mà không cần tải lại trang (chống giật lag và tiết kiệm băng thông).
- **Hệ điều hành thời gian thực (FreeRTOS):** Toàn bộ hệ thống Web Server được đóng gói và chạy trên một luồng (Task) hoàn toàn riêng biệt có tên là `main_server_task`. Kiến trúc này cho phép các chức năng cốt lõi khác của hệ thống (như đọc cảm biến môi trường, xử lý tín hiệu, hay chạy mô hình AI TinyML) có thể hoạt động song song ở các Task khác mà không bị chặn (blocking) bởi các tiến trình mạng.

3.5.2 Quản lý mạng Wi-Fi và NVS (Non-Volatile Storage)

Hệ thống sử dụng linh hoạt hai chế độ mạng của vi điều khiển ESP32 bao gồm **AP (Access Point - trạm phát)** và **STA (Station - thiết bị kết nối)**:

- **AP Mode:** Chế độ này luôn được kích hoạt mặc định khi hệ thống khởi động (thông qua hàm `startAP()`). ESP32 tự đóng vai trò là một trạm phát Wi-Fi độc lập, cho phép thiết bị của người dùng kết nối trực tiếp vào để tiến hành cấu hình mạng ban đầu.
- **STA Mode:** Khi người dùng cung cấp thông tin mạng cục bộ (SSID và Mật khẩu) từ giao diện, thiết bị sẽ chuyển sang chế độ `WIFI_AP_STA` (hoạt động song song cả thu và phát). Nếu kết nối thành công tới router nội bộ, hệ thống sẽ ghi nhận và hiển thị địa chỉ IP được cấp phát.
- **Lưu trữ Preferences (NVS):** Để đảm bảo tính tự động hóa, hệ thống sử dụng thư viện `<Preferences.h>` nhằm lưu trữ vĩnh viễn dữ liệu cấu hình mạng vào phân vùng Flash (Non-Volatile Storage). Ở những lần khởi động tiếp theo, vi điều khiển sẽ tự động gọi hàm `loadWifiCredentials()` để trích xuất thông tin này và tái thiết lập kết nối mà không cần sự can thiệp của người dùng.



Hình 5: Kiến trúc tổng thể của hệ thống Web Server trên ESP32.

3.5.3 Backend Web Server (API và Routing)

Hệ thống sử dụng thư viện `<WebServer.h>` để định nghĩa các Endpoints (đường dẫn API). Cụ thể, trong hàm `setupServer()`, máy chủ web được cấu hình với 3 nhóm API cốt lõi:

1. **Phục vụ giao diện (HTML):** Các đường dẫn `/`, `/dashboard`, và `/settings`. Nhóm này có nhiệm vụ trả về toàn bộ chuỗi HTML (chứa CSS/JS) để hiển thị giao diện Single Page Application trên trình duyệt.

2. **API điều khiển & cấu hình:** Bao gồm `/toggle`, `/led-mode`, `/led-rgb`, `/connect`, và `/scan-networks`. Đây là các endpoint tiếp nhận lệnh từ người dùng để thay đổi trạng thái phần cứng (bật/tắt, đổi màu LED) hoặc tiến hành quét/kết nối mạng Wi-Fi.

3. **API cung cấp dữ liệu (JSON):** Bao gồm `/sensors`, `/history`, `/led-status`, và `/tinyml-status`. Giao diện web sẽ liên tục gọi các API này để cập nhật số liệu. Đặc biệt, thông qua hàm `handleTinyMlStatus()`, ta có thể thấy rõ cơ chế giao tiếp liên luồng (inter-task communication) giữa Web Task và AI Task thông qua việc đọc các biến toàn cục (ví dụ: `tinyml_predicted_label`).

3.5.4 Giao diện Frontend (HTML, CSS và JavaScript)

Thay vì sử dụng các tệp tin lưu trữ rời rạc, toàn bộ giao diện người dùng được nhúng trực tiếp vào mã nguồn dưới dạng các chuỗi ký tự định dạng `rawliteral`. Kiến trúc giao diện tập trung vào các đặc điểm sau:

- **Single Page Application (SPA):** Dashboard được thiết kế để hoạt động trên một trang duy nhất. Sử dụng thư viện `Chart.js` để trực quan hóa dữ liệu cảm biến. Giao diện có khả năng phản hồi (Responsive) tốt trên cả thiết bị di động và máy tính.

- **Cơ chế Fallback Canvas:** Đây là một điểm sáng trong thiết kế hệ thống. Trong điều kiện thiết bị không có kết nối Internet (không thể tải thư viện từ CDN), mã nguồn JavaScript sẽ tự động kích hoạt hàm `drawFallbackChart()` sử dụng `Canvas API` thuần để vẽ biểu đồ. Điều này đảm bảo tính sẵn sàng cao của hệ thống trong mọi môi trường mạng.

- **Cập nhật dữ liệu thời gian thực:** Sử dụng hàm `setInterval()` để tự động thực hiện các truy vấn `fetch()` tới các endpoint JSON (`/history`, `/led-status`, `/tinyml-status`) định kỳ từ 3 đến 5 giây, giúp cập nhật trạng thái thiết bị và dự đoán từ mô hình TinyML mà không làm gián đoạn trải nghiệm người dùng.

3.5.5 Quản lý tác vụ và Luồng xử lý (Task Management)

Tiến trình cốt lõi của máy chủ được thực thi bên trong hàm `main_server_task`, vận hành dựa trên các nguyên tắc của hệ điều hành thời gian thực FreeRTOS:

- **Vòng lặp điều phối (Main Loop):** Task duy trì một vòng lặp vô hạn để xử lý đồng thời các yêu cầu phân giải tên miền (`dnsServer.processNextRequest()`) và các yêu cầu từ máy khách HTTP (`server.handleClient()`).
  - **Cơ chế Non-blocking:** Logic kết nối Wi-Fi được thiết kế không chặn (non-blocking) bằng cách sử dụng hàm `millis()` để kiểm soát thời gian chờ (timeout 10 giây), giúp Web Server vẫn có thể đáp ứng các yêu cầu khác trong khi đang đợi thiết lập kết nối mạng.
  - **Tương tác phím cứng:** Hệ thống liên tục giám sát trạng thái của `BOOT_PIN`. Khi người dùng nhấn giữ nút này, thiết bị sẽ cưỡng bức chuyển về chế độ AP, giúp người dùng luôn có khả năng truy cập vào trang cấu hình kể cả khi mạng cục bộ gặp sự cố.
- **Giao tiếp liên luồng (Inter-task Communication):** Sau khi kết nối Wi-Fi thành công, hệ thống sử dụng cơ chế Semaphore (`xSemaphoreGive`) để gửi tín hiệu cho các Task khác biết rằng kết nối Internet đã sẵn sàng, cho phép kích hoạt các tính năng nâng cao như đồng bộ hóa dữ liệu lên Cloud.

3.6 Task 5: Triển khai TinyML và đánh giá độ chính xác

3.6.1 Tổng quan luồng thực thi của TinyML:

Toàn bộ hệ thống được thiết kế theo một luồng khép kín từ môi trường vật lý đến mô hình toán học và quay trở lại thiết bị nhúng. Kiến trúc này được chia thành ba giai đoạn (Phase) chiến lược, đảm bảo tính minh bạch và tối ưu hóa tài nguyên cho vi điều khiển ESP32.

Phase 1: Luồng dữ liệu (Data Pipeline)

Giai đoạn này đóng vai trò là "cầu nối"chuyển đổi các tín hiệu vật lý thành tri thức số học.

- **Số hóa tín hiệu:** Dữ liệu điện tử từ cảm biến được thu thập qua cổng Serial dưới dạng log thô thông qua `collect_from_log.py`.
- **Cấu trúc hóa và Gán nhãn:** Trọng tâm của giai đoạn này là biến các dòng văn bản rời rạc thành tập dữ liệu cấu trúc (`.csv`). Sử dụng `extract_ready_data.py`, hệ thống áp dụng các quy tắc logic nghiệp vụ (Heuristics) để tự động gán nhãn cho từng cặp giá trị (Nhiệt độ, Độ ẩm), tạo ra "Ground Truth"cho quá trình huấn luyện AI.

Phase 2: Luồng Học máy (ML Pipeline)

Được ví như "bộ não"của hệ thống, nơi diễn ra quá trình chọn lọc tự nhiên giữa các thuật toán học máy.

- **Bộ lọc ESP32 Score:** Thay vì chỉ tối ưu hóa độ chính xác, hệ thống sử dụng một bộ chỉ số đánh giá khắt khe trong `evaluate_models_comprehensive.py`. Mô hình được chọn phải thỏa mãn sự cân bằng giữa độ chính xác, tốc độ suy luận và kích thước bộ nhớ.
- **White-box Export:** Điểm khác biệt cốt lõi là việc không sử dụng các tệp mô hình `.tflite` nặng nề. Thay vào đó, `export_naive_bayes.py` trích xuất các tham số xác suất (Theta, Variance) và dịch chúng trực tiếp thành ngôn ngữ C (`.h`), cho phép thực thi suy luận bằng các phép toán cơ bản.

Phase 3: Triển khai và Xác thực (Deployment & Validation)

Giai đoạn cuối cùng thực hiện việc thực thi mô hình đồng thời trên hai môi trường để đảm bảo tính nhất quán.

- **Nhánh Kiểm thử (PC):** `test_naive_bayes.py` sử dụng các tham số đã xuất để giả lập suy luận trên máy tính, giúp kiểm tra tính đúng đắn của thuật toán trước khi nạp vào phần cứng.

- **Nhánh Thực thi (ESP32):** Mô hình được tích hợp vào tác vụ `tiny_ml_task` trong `tinyml.cpp`. Tại đây, vi điều khiển sử dụng chung "hệ quy chiếu toán học"với máy tính để đưa ra quyết định tại chỗ (At-the-edge) dựa trên dữ liệu cảm biến thời gian thực.

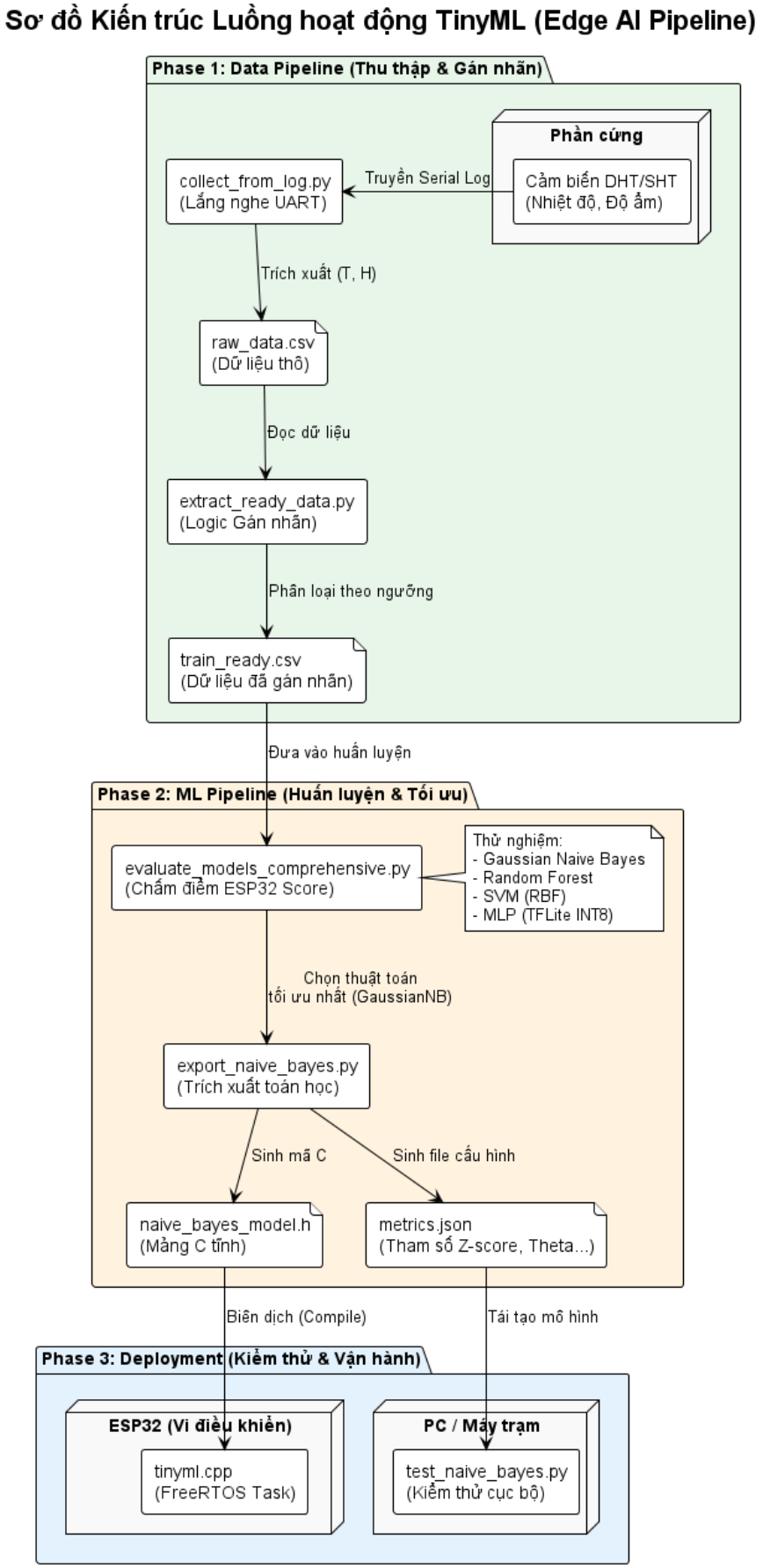
Sự phối hợp giữa ba giai đoạn này tạo nên một hệ thống TinyML tinh gọn, có khả năng vận hành bền bỉ trên các thiết bị IoT hạn chế về tài nguyên.

3.6.2 Thu thập dữ liệu từ cảm biến (Data Collection)

Giai đoạn đầu tiên của hệ thống là thu thập dữ liệu thô từ cảm biến nhiệt độ và độ ẩm thông qua giao tiếp Serial. `collect_from_log.py` đóng vai trò là trình lắng nghe và xử lý dữ liệu từ vi điều khiển ESP32.

- **Giao tiếp Serial:** Sử dụng thư viện `pyserial` để thiết lập kết nối UART với Baudrate mặc định là 115200. Hệ thống liên tục đọc các dòng log từ cổng chỉ định (ví dụ: COM4).
- **Cơ chế phân tích (Parsing):** Để đảm bảo tính linh hoạt, hệ thống hỗ trợ ba định dạng đầu vào khác nhau thông qua hàm `parse_temp_humi_from_line`:
  1. **JSON:** Phân tích chuỗi có dạng `{"temperature": X, "humidity": Y}`.
  2. **Regex:** Sử dụng biểu thức chính quy `T\s*=\s*(...)\D+H\s*=\s*(...)` để tìm các mẫu như "T=30.5 H=65.2".
  3. **CSV Fallback:** Nếu không khớp các định dạng trên, hệ thống sẽ tự động tách chuỗi bằng dấu phẩy và lấy hai giá trị cuối cùng.
- **Lưu trữ dữ liệu:** Dữ liệu sau khi trích xuất thành công được gắn nhãn thời gian thực (ISO 8601) và ghi nối tiếp vào tệp `raw_data.csv` bằng `csv.DictWriter`.

Việc sử dụng `f.flush()` sau mỗi dòng ghi giúp đảm bảo dữ liệu không bị mất mát trong trường hợp kết nối Serial bị ngắt đột ngột.



Hình 6: Tổng quan luồng thực thi của TinyML từ thu thập dữ liệu đến triển khai trên ESP32.

3.6.3 Tiền xử lý và Gán nhãn dữ liệu (Data Preprocessing and Labeling)

Sau khi thu thập được dữ liệu thô, bước tiếp theo là làm sạch và gán nhãn tự động để tạo ra tập dữ liệu sẵn sàng cho việc huấn luyện mô hình (train-ready). Quá trình này được thực hiện thông qua `extract_ready_data.py`.

- **Mục tiêu:** Chuyển đổi tệp `raw_data.csv` thành `train_ready.csv` bằng cách áp dụng các quy tắc logic nghiệp vụ để xác định trạng thái của hệ thống dựa trên thông số môi trường.
- **Quy tắc gán nhãn (Heuristic Labeling):** Hệ thống sử dụng hàm `classify_label` để phân loại dữ liệu vào các nhãn mục tiêu:
  - **Bật máy sưởi:** Khi nhiệt độ thấp hơn 24.0°C.
  - **Bật điều hòa:** Khi nhiệt độ lớn hơn 33.0°C và độ ẩm nằm trong khoảng 50% - 80%.
  - **Bật máy hút ẩm:** Khi nhiệt độ lớn hơn 33.0°C và độ ẩm vượt quá 80%.
  - **Không hợp lệ:** Dữ liệu nằm ngoài dải vật lý (0 - 100) hoặc thiếu giá trị.
  - **Chế độ bình thường:** Các trường hợp còn lại trong dải đo cho phép.

Kết quả của giai đoạn này là một tệp CSV gồm ba cột chính: `temperature`, `humidity`, và `label`, phục vụ trực tiếp cho các thuật toán học máy ở bước sau.

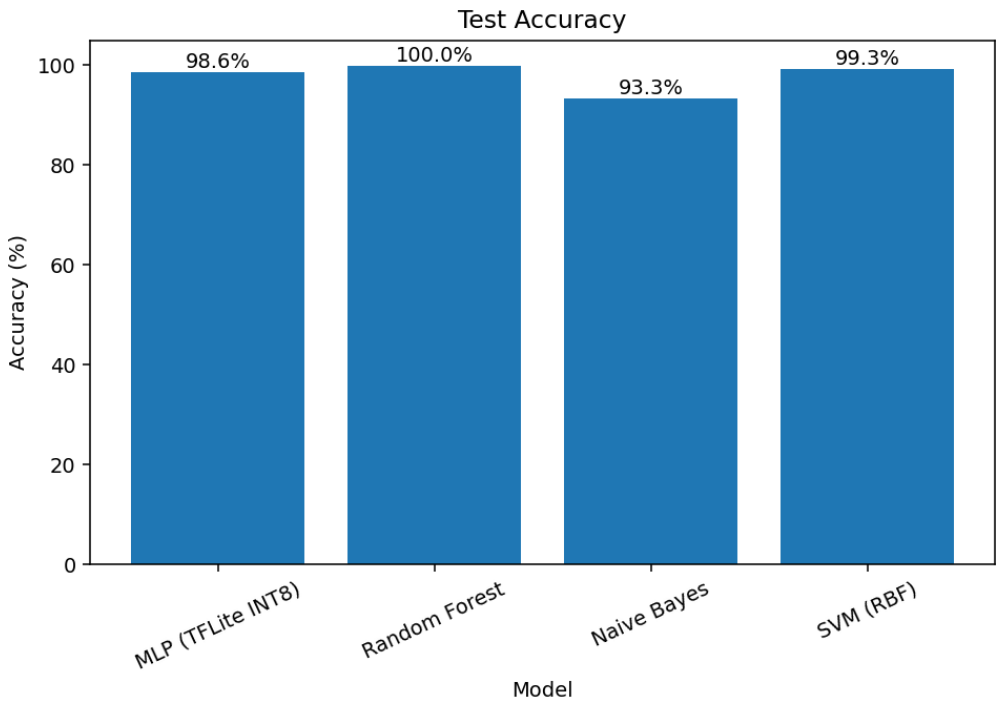
3.6.4 Đánh giá và So sánh Mô hình (Model Benchmarking)

Để tìm ra kiến trúc học máy phù hợp nhất cho phần cứng hạn chế tài nguyên, hệ thống tiến hành đánh giá toàn diện nhiều thuật toán khác nhau thông qua `evaluate_models_comprehensive.py`. Quá trình này không chỉ xét đến độ chính xác mà còn đặc biệt chú trọng vào tài nguyên hệ thống.

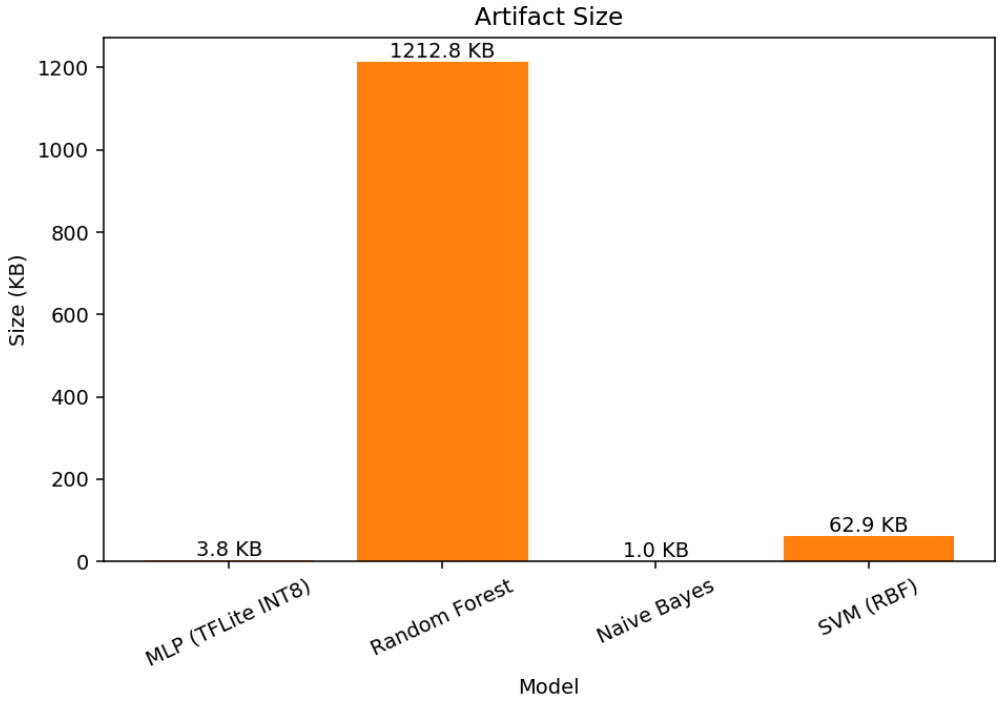
- **Các mô hình thử nghiệm:** Hệ thống tiến hành huấn luyện và đối chiếu bốn mô hình đại diện:
  1. Gaussian Naive Bayes (GaussianNB).
  2. Random Forest.
  3. Support Vector Machine (SVM) với nhân RBF.
  4. Multi-Layer Perceptron (MLP) - Kích thước được ước lượng bằng cách lượng tử hóa trọng số về INT8 theo chuẩn TensorFlow Lite.
- **Chỉ số đánh giá ML tiêu chuẩn:** Các mô hình được đo lường hiệu suất thông qua Độ chính xác (Accuracy), Precision, Recall, F1-Score (trung bình có trọng số) và diện tích dưới đường cong ROC (ROC-AUC). Hệ thống cũng sử dụng K-Fold Cross-validation (với $K = 5$) để đánh giá tính ổn định của mô hình.

- **Chỉ số phần cứng (ESP32 Score):** Điểm đột phá của luồng công việc này là việc định nghĩa một công thức tính điểm (thang 100) dành riêng cho TinyML, bao gồm ba trọng số chính:
  - **Độ chính xác (40%):** Ưu tiên khả năng nhận diện đúng trạng thái.
  - **Kích thước mô hình (30%):** Dung lượng tính bằng KB. Mô hình < 100KB được coi là lý tưởng; mô hình > 1000KB bị trừ điểm mạnh do giới hạn bộ nhớ Flash/RAM của vi điều khiển.
  - **Tốc độ suy luận (30%):** Đánh giá thời gian dự đoán (ms) để đảm bảo đáp ứng thời gian thực.

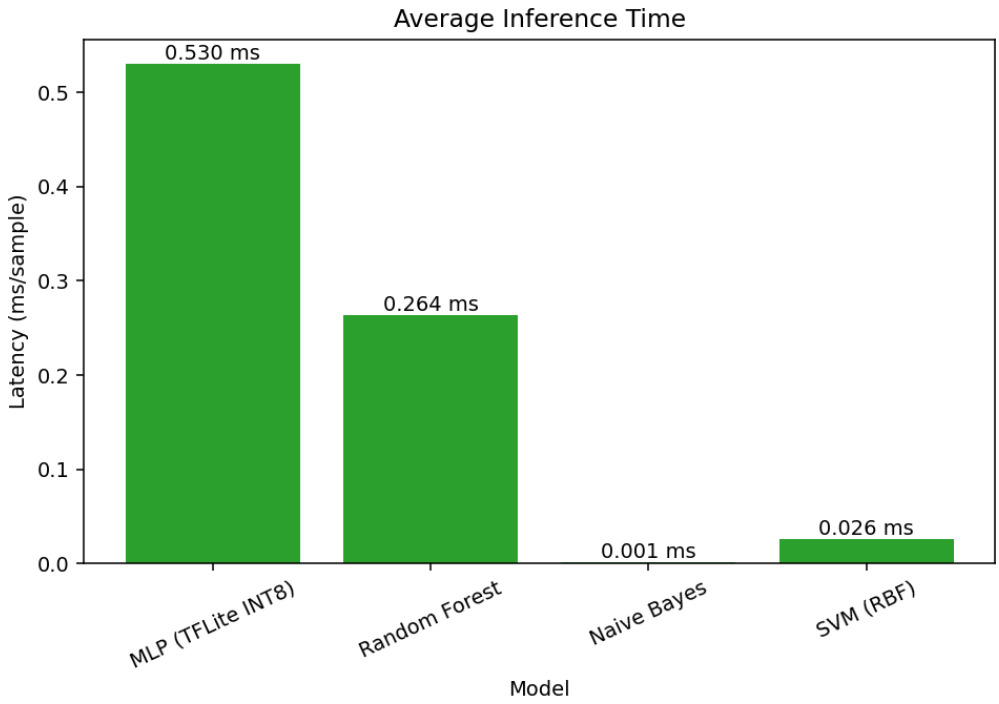
Kết thúc quá trình đánh giá, hệ thống kết xuất một bảng xếp hạng (Ranking) và báo cáo JSON chi tiết. Các biểu đồ so sánh trực quan (như *Model Size Comparison* và *ESP32 Score*) sẽ tự động được tạo ra nhằm hỗ trợ lập trình viên ra quyết định lựa chọn thuật toán triển khai cuối cùng.



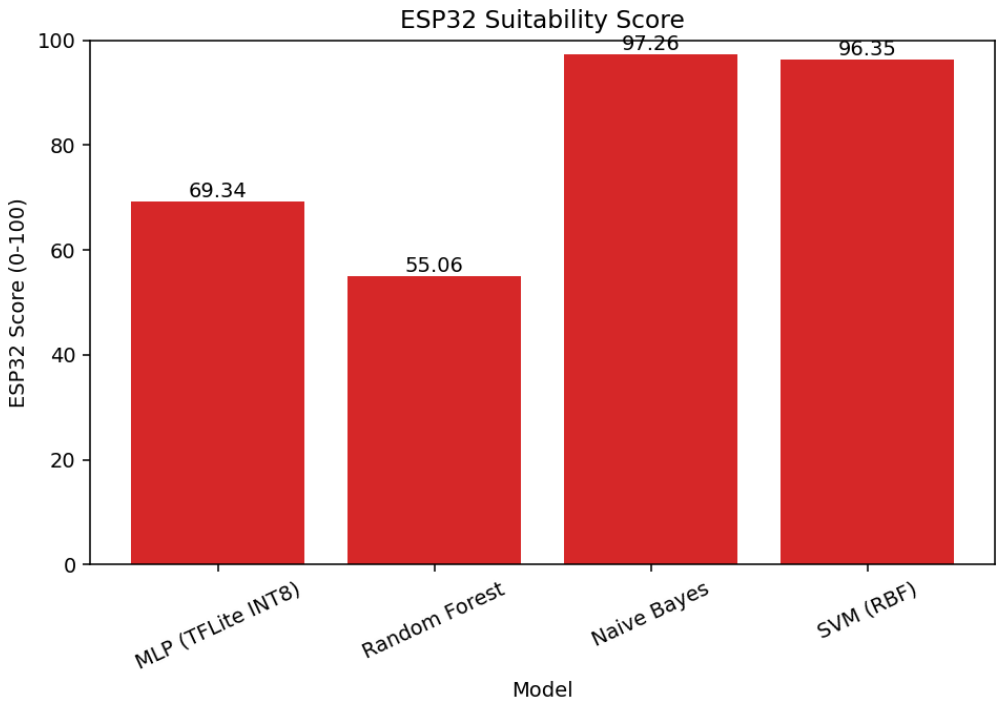
Hình 7: Biểu đồ so sánh độ chính xác của các mô hình ML khác nhau trên tập dữ liệu huấn luyện.



Hình 8: Biểu đồ so sánh kích thước mô hình của các mô hình ML khác nhau.



Hình 9: Biểu đồ so sánh thời gian suy luận của các mô hình ML.



Hình 10: Biểu đồ so sánh điểm số ESP32 của các mô hình ML.

3.6.5 Huấn luyện và Xuất mã C (Model Training and C Header Export)

Dựa trên kết quả đánh giá ở giai đoạn trước, thuật toán Gaussian Naive Bayes (GaussianNB) được lựa chọn nhờ sự cân bằng tối ưu giữa độ chính xác và yêu cầu tài nguyên siêu nhỏ. Thay vì sử dụng các framework biên dịch nặng nề như TensorFlow Lite (TFLite) cho vi điều khiển, hệ thống thực thi một phương pháp triển khai trực tiếp thông qua `export_naive_bayes.py`.

- **Cách triển khai:** GaussianNB không phải là một mạng nơ-ron tiêu chuẩn, do đó việc xuất sang định dạng TFLite không phải là giải pháp phù hợp. Thay vào đó, mô hình được "đóng băng"bằng cách trích xuất các tham số toán học đã học thành các mảng hằng số ngôn ngữ C (C arrays) và tự hiện thực hàm suy luận (score function) trực tiếp trên firmware của ESP32.

- **Chuẩn hóa Z-score:** Trước khi huấn luyện, dữ liệu đầu vào được chuẩn hóa sử dụng phương pháp Z-score. Các tham số trung bình (`mean`) và độ lệch chuẩn (`std`) được lưu lại để hệ thống nhúng có thể tiền xử lý dữ liệu cảm biến thực tế theo đúng tỷ lệ toán học.

- **Trích xuất ma trận cốt lõi:** Mô hình được huấn luyện bằng thư viện `scikit-learn`, sau đó trích xuất các thông số phân phối chuẩn bao gồm:
  - Xác suất tiên nghiệm của từng lớp (`class_prior_`).
  - Kỳ vọng (`theta_`) và Phương sai (`var_`) của từng đặc trưng (Nhiệt độ, Độ ẩm) tương ứng với từng trạng thái hệ thống.

Cấu trúc tệp tiêu đề C (C Header Output)

Kết quả của quá trình xuất là tệp `naive_bayes_model.h`, chứa toàn bộ tri thức của AI dưới dạng các mảng bộ nhớ tĩnh tối ưu cho SRAM của ESP32.

```
#ifndef NAIVE_BAYES_MODEL_H
#define NAIVE_BAYES_MODEL_H

#define NB_NUM_CLASSES 5
#define NB_NUM_FEATURES 2

static const float kNbFeatureMean[NB_NUM_FEATURES] = {47.44557571f, 50.07275391f};
static const float kNbFeatureStd[NB_NUM_FEATURES] = {30.14743614f, 28.64877701f};
// ... (Chi tiet cac mang kNbClassPrior, kNbTheta, kNbVariance) ...

// Runtime formula on ESP32:
// 1) normalize input: x_norm[j] = (x[j] - kNbFeatureMean[j]) / kNbFeatureStd[j]
// 2) score[class] = log(prior) - 0.5 * sum(log(2*pi*var)) - 0.5 *
    sum((x_norm-theta)^2 / var)
// 3) choose argmax(score)
#endif
```

Điểm nhấn kỹ thuật nằm ở phần chú thích cuối tệp. Nó cung cấp chính xác công thức toán học tính toán hàm Log-Likelihood, giúp lập trình viên nhúng dễ dàng dịch ngược (reverse-engineer) thuật toán ML thành mã C/C++ thuần túy mà không cần phải gọi đến bất kỳ thư viện hỗ trợ học máy nào bên ngoài.

3.6.6 Kiểm thử mô hình cục bộ (Local Model Testing)

Bước cuối cùng trước khi triển khai firmware lên phần cứng thực tế là kiểm chứng lại logic suy luận. Việc gỡ lỗi (debug) số học trực tiếp trên vi điều khiển thường tốn thời gian và khó khăn, do đó, `test_naive_bayes.py` được thiết kế để giả lập lại chính xác môi trường tính toán của ESP32 ngay trên máy tính cá nhân.

- **Đồng bộ tham số (Parameter Synchronization):** Hệ thống không tải lại mô hình từ thư viện `scikit-learn`. Thay vào đó, nó đọc trực tiếp tệp `naive_bayes_model_metrics.json` (được sinh ra ở phần trước) chứa các ma trận như `zscore_mean`, `zscore_std`, `theta`, và `variance`. Điều này đảm bảo Python và C++ sử dụng chung một "bộ não"tham số.
- **Tái tạo logic toán học:** Hệ thống thực hiện lại các bước tính toán giống hệt mã C sẽ chạy trên ESP32: chuẩn hóa Z-score đầu vào và tính Log-Likelihood dựa trên phân phối chuẩn Gaussian.

- **Đánh giá độ tin cậy (Confidence Scoring):** Trong khi vi điều khiển có thể chỉ cần hàm `argmax()` để tìm nhãn có điểm cao nhất nhằm tiết kiệm chu kỳ CPU, PC bổ sung thêm hàm `softmax` để chuyển đổi các giá trị log-likelihood thô thành xác suất phần trăm (%). Điều này giúp lập trình viên đánh giá độ "tự tin"của mô hình đối với từng dự đoán.
- **Giao diện tương tác (Interactive CLI):** Chương trình cung cấp giao diện dòng lệnh cho phép người dùng nhập tay các giá trị Nhiệt độ và Độ ẩm bất kỳ, sau đó in ra dự đoán tốt nhất (Top 1) kèm theo danh sách xếp hạng các khả năng khác (Top K).

3.6.7 Triển khai Firmware trên Vi điều khiển (On-device Deployment)

Sau khi mô hình được xuất sang mảng C và kiểm chứng logic thành công, bước cuối cùng là tích hợp "bộ não"này vào hệ điều hành thời gian thực (FreeRTOS) trên ESP32. Tệp mã nguồn `tinyml.cpp` đảm nhiệm vai trò là cầu nối giữa dữ liệu cảm biến thời gian thực và thuật toán Naive Bayes.

- **Kiến trúc đa tác vụ (Multitasking với FreeRTOS):** Thay vì chạy tuần tự trong vòng lặp `loop()` chặn (blocking) các tiến trình khác, hệ thống gói gọn quá trình suy luận vào một tác vụ độc lập mang tên `tiny_ml_task`.
  - Tác vụ này sử dụng cơ chế **Semaphore** (`xBinarySemaphoreTinyMLData`) để đồng bộ hóa. Nó sẽ ở trạng thái "ngủ"(Blocked) và không tiêu tốn chu kỳ CPU cho đến khi tác vụ đọc cảm biến (`temp_humi_monitor`) báo hiệu có dữ liệu mới.
- **Động cơ suy luận (Inference Engine):** Hàm `runNaiveBayes` hiện thực hóa lại chính xác các phương trình toán học đã đề cập ở Phần 4 và Phần 5 bằng thư viện `<math.h>` của ngôn ngữ C:
  1. **Chuẩn hóa:** Dữ liệu từ cảm biến thực (`glob_temperature`, `glob_humidity`) được chuẩn hóa Z-score qua hàm `normalizeInput` sử dụng hằng số `kNbFeatureMean` và `kNbFeatureStd`.
  2. **Tính toán Log-Likelihood:** Sử dụng vòng lặp để cộng dồn xác suất theo công thức phân phối Gaussian.
  3. **Hàm Softmax & Argmax:** Tính toán xác suất (%) cho từng lớp (`class_probs`) và chọn ra nhãn có xác suất cao nhất (`argmax`) làm kết quả dự đoán.
- **Cơ chế đánh giá On-device (On-the-fly Evaluation):** Một điểm sáng trong thiết kế phần mềm này là khối lệnh `tryUpdateGroundTruthFromSerial`. Nó biến ESP32 thành một công cụ tự đánh giá ngoài hiện trường. Thông qua cổng Serial, lập trình viên có thể gõ các lệnh như `gt 1` (gán nhãn đối chứng thực tế là 1) để ESP32 tự động đối chiếu dự đoán của nó với thực tế, từ đó thống kê số lượng dự đoán đúng/tổng số (`eval_correct` / `eval_total`).

Mã nguồn cốt lõi: Vòng lặp suy luận thời gian thực

Đoạn lệnh dưới đây minh họa cách mô hình ML hoạt động trơn tru trong môi trường RTOS mà không làm gián đoạn các luồng nghiệp vụ khác của hệ thống nhúng:

```
void tiny_ml_task(void *pvParameters) {
    setupTinyML();
    while (1) {
        // Kiem tra cong Serial de cap nhat nhan doi chung (neu co)
        tryUpdateGroundTruthFromSerial();

        // Cho doi tin hieu (Semaphore) tu cam bien
        if (xSemaphoreTake(xBinarySemaphoreTinyMLData, pdMS_TO_TICKS(10000)) !=
            pdTRUE) {
            continue;
        }

        // Chay suy luan AI
        runNaiveBayes(glob_temperature, glob_humidity);
        const int predicted_label = argmax(class_probs, NB_NUM_CLASSES);

        // Cap nhat bien toan cuc va in ket qua
        tinyml_predicted_label = predicted_label;
        Serial.print("T="); Serial.print(glob_temperature, 2);
        Serial.print("␣H="); Serial.print(glob_humidity, 2);
        Serial.print("␣|␣Label="); Serial.println(kNbLabels[predicted_label]);

        // Thong ke do chinh xac tai cho (neu bat Ground Truth)
        if (has_ground_truth) {
            ++eval_total;
            if (predicted_label == ground_truth_label) ++eval_correct;
        }

        vTaskDelay(kInferencePeriodTicks); // Chu ky suy luan (5 giay)
    }
}
```

Toàn bộ quá trình từ thu thập, huấn luyện, xuất thông số cho đến tính toán trên vi điều khiển hoàn toàn minh bạch, không phụ thuộc vào hộp đen (black-box API) nào, tối ưu hóa đến từng byte RAM và chu kỳ xung nhịp của vi xử lý.

3.7 Task 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT

3.7.1 Nguyên lý hoạt động

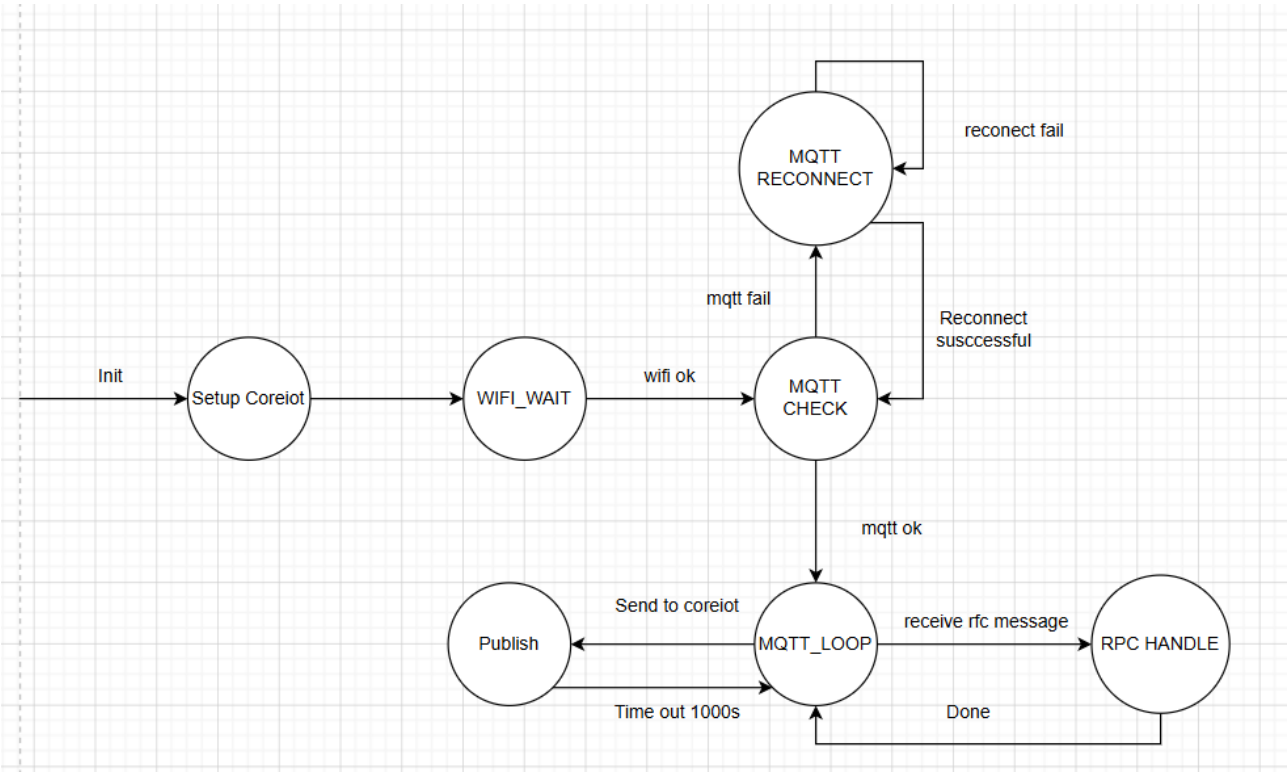
Tác vụ "CoreIoT"đóng vai trò là cầu nối giao tiếp hai chiều (Bidirectional Communication) giữa biên thiết bị (ESP32) và nền tảng đám mây (Cloud/ThingsBoard) thông qua giao thức MQTT ở port 1883. Tác vụ này thực hiện hai nhiệm vụ cốt lõi: báo cáo dữ liệu định kỳ và lắng nghe, thực thi các lệnh gọi hàm từ xa (RPC - Remote Procedure Call) từ người dùng.

- **Quản lý kết nối không xung đột:** Để tránh hiện tượng "tranh chấp"module Wi-Fi với Web Server gây treo hệ thống, Task CoreIoT không tự gọi hàm `WiFi.begin()`. Thay vào đó, nó liên tục giám sát cờ `isWifiConnected` do `main_server_task` cấp phát. Nếu mất Wi-Fi, tác vụ tự động chuyển sang trạng thái chờ 2000ms; chỉ khi có mạng, quá trình kết nối MQTT (`reconnect`) mới được phép kích hoạt.

- **Đồng bộ:** Khi duy trì kết nối MQTT ổn định, cứ mỗi 10.000ms (10 giây), hệ thống sẽ đóng gói các thông số môi trường hiện hành (`glob_temperature`, `glob_humidity`) và cấu hình ngưỡng cảnh báo (`glob_temp_threshold`) thành chuỗi JSON. Chuỗi này được gửi lên topic `v1/devices/me/telemetry` để hiển thị lên Dashboard của hệ thống CoreIoT.

- **Thực thi lệnh điều khiển RPC (RPC Callback):** Hệ thống đăng ký (subscribe) lắng nghe tại topic `v1/devices/me/rpc/request/+`. Khi có lệnh điều khiển từ Dashboard (VD: kéo Slider thay đổi ngưỡng), hàm `callback` sẽ được ngắt để gọi. Thư viện `ArduinoJson` phân tích gói tin, nhận diện phương thức `"setThre"` và ngay lập tức trích xuất tham số `params` để cập nhật đè lên biến `glob_temp_threshold`. Nhờ đó, thay đổi từ Cloud lập tức được phản ánh xuống giới hạn cảnh báo của màn hình LCD ở biên thiết bị.

3.7.2 Sơ đồ máy trạng thái (FSM)



Hình 11: Task 6 FSM

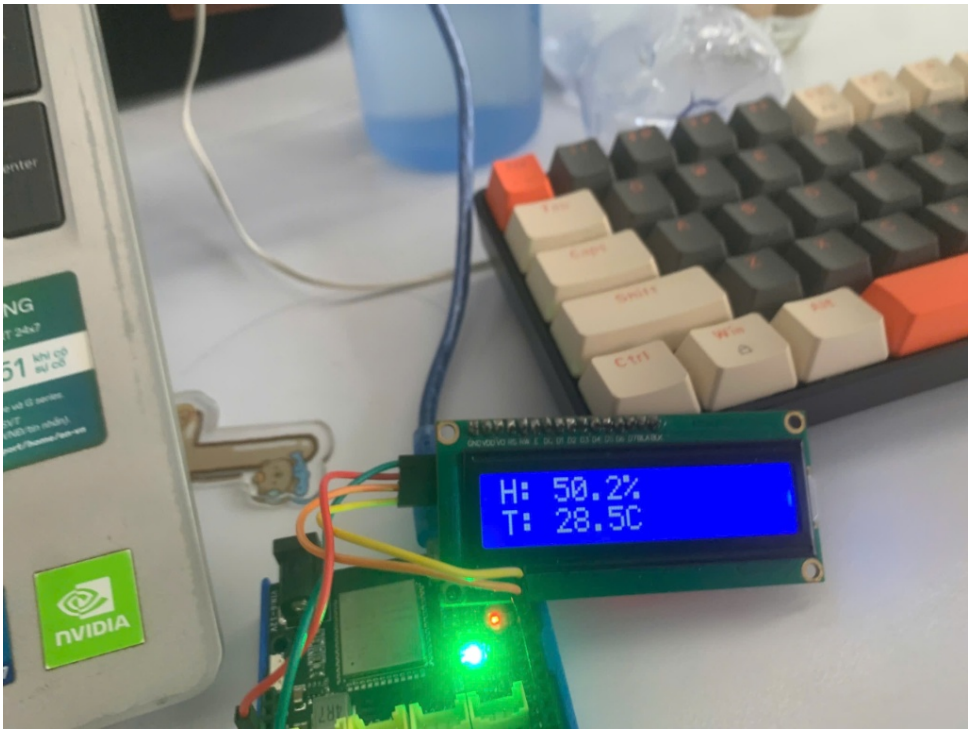
3.8 Tóm tắt chương

Chương này tổng kết quá trình triển khai hệ thống trên ESP32. Thông qua FreeRTOS, hệ thống đã vận hành đa nhiệm đồng bộ các module phần cứng: cảm biến DHT20, màn hình LCD và hệ thống LED cảnh báo trực quan. Nền tảng mạng được thiết lập linh hoạt, kết hợp Máy chủ Web cục bộ để cấu hình thiết bị và giao tiếp đám mây CoreIoT qua giao thức MQTT. Đặc biệt, việc nhúng thành công mô hình học máy TinyML tại biên đã nâng cấp thiết bị thành một hệ thống AIoT thông minh, tự chủ suy diễn dữ liệu môi trường theo thời gian thực.

4 Đánh giá và kết quả thực nghiệm

4.1 Kết quả chức năng

4.1.1 Task 1 & 2: LED và NeoPixel



- Led và NeoPixel phản hồi với nhiệt độ độ ẩm đọc từ cảm biến bằng màu sắc

4.1.2 Task 3: Giám sát môi trường qua LCD

- **Hiển thị thời gian thực:** Ở chế độ thông thường, màn hình cập nhật chính xác các thông số môi trường
- **Cơ chế ngắt cảnh báo:** Khi xảy ra sự kiện quá nhiệt (28.4°C > 25°C), màn hình lập tức xóa dữ liệu cũ và thực hiện hiệu ứng chớp tắt dòng chữ `! WARNING TEMP !` ở hàng trên.

4.1.3 Task 4: Máy chủ Web cục bộ (Local Web Server)

Hình 12: Scan wifi kết nối sang chế độ STA

Hình 13: Sau kết nối wifi thành công

- **Cấu hình mạng (AP Mode):** Cung cấp giao diện "Cấu hình Wi-Fi", cho phép người dùng quét và nhập mật khẩu kết nối mạng nội bộ. Sau khi kết nối thành công với wifi sẽ cung một địa chỉ IP mới ở chế độ STA.
- **Giám sát nội bộ (SPA Dashboard):** Giao diện Web hiển thị số liệu trực quan với biểu đồ thời gian thực cho cả Nhiệt độ và Độ ẩm. Các khối chức năng điều khiển chế độ LED (AUTO/MANUAL) và tinh chỉnh màu sắc đều hiển thị.

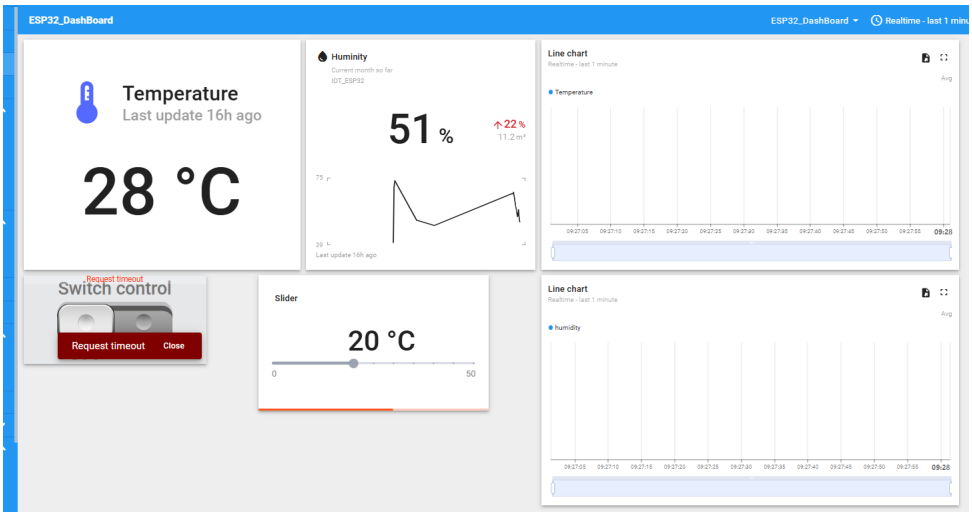
4.1.4 Task 5: (TinyML)

Thuật toán Naive Bayes được nhúng thành công và có khả năng đưa ra các suy diễn logic ngay trên phần cứng hạn chế của ESP32:

- Thông qua giao diện Web, hệ thống trả về kết quả dự đoán "Chế độ bình thường"(Nhãn 4) với độ tin cậy (Confidence) đạt 72.0

- Dựa trên kết quả này, hệ thống chủ động đưa ra khuyến nghị: "Điều kiện ổn định, có thể giữ chế độ hiện tại". Điều này chứng minh tính thực tiễn của Edge AI trong việc hỗ trợ ra quyết định.

4.1.5 Task 6: Tích hợp đám mây CoreIoT



Hình 14: Cấu hình ngưỡng nhiệt độ trên CoreIot

Hình 15: Phản hồi thiết bị với ngưỡng nhiệt độ mới

- **Đồng bộ (Telemetry):** Dashboard trên ThingsBoard nhận và trực quan hóa dữ liệu ổn định

- **Điều khiển RPC và Xử lý ngoại lệ:** Thanh Slider thiết lập ngưỡng nhiệt độ (setThre) hoạt động (ang ở mức 20°C.

4.2 Đánh giá TinyML

Quá trình đánh giá mô hình đã chứng minh tính khả thi của việc triển khai trí tuệ nhân tạo trực tiếp trên thiết bị biên (Edge AI) với tài nguyên phần cứng vô cùng hạn chế. Thay vì chỉ phụ thuộc vào các chuẩn đo lường học máy truyền thống, hệ thống đã áp dụng bộ tiêu chí đánh giá khắt khe dành riêng cho vi điều khiển (ESP32 Score), tập trung cân bằng giữa ba yếu tố cốt lõi: Độ chính xác của dự đoán, Kích thước bộ nhớ tiêu thụ, và Tốc độ suy luận thời gian thực.

4.3 Tóm tắt chương

Chương này đã trình bày các kết quả kiểm thử và đánh giá của hệ thống sau khi hoàn thiện. Ở phần kết quả chức năng, các chức năng chính của hệ thống đã được kiểm tra để xác nhận tính ổn định, khả năng hoạt động đúng theo yêu cầu và mức độ đáp ứng trong thực tế. Bên cạnh đó, phần đánh giá TinyML cho thấy mô hình có thể được triển khai hiệu quả trên thiết bị biên với tài nguyên hạn chế, đồng thời vẫn bảo đảm được sự cân bằng giữa độ chính xác, dung lượng bộ nhớ sử dụng và tốc độ suy luận. Từ các kết quả đạt được, có thể khẳng định giải pháp đề xuất phù hợp với định hướng ứng dụng IoT kết hợp AI nhúng và có tính khả thi khi triển khai thực tế.

5 Kết luận

- **Tối ưu hóa đa nhiệm:** Ứng dụng hệ điều hành FreeRTOS để phân luồng và đồng bộ trơn tru các tác vụ phần cứng bao gồm cảm biến, LCD và hệ thống cảnh báo trực quan.
- **Kiến trúc mạng linh hoạt:** Tích hợp thành công Máy chủ Web nội bộ để cấu hình thiết bị và luồng giao tiếp MQTT kết nối đám mây CoreIoT, đảm bảo khả năng giám sát, điều khiển hai chiều theo thời gian thực.
- **Tích hợp Edge AI:** Nhúng thành công mô hình học máy TinyML (Naive Bayes) tại biên, giúp thiết bị tự chủ suy diễn dữ liệu và đưa ra khuyến nghị tức thời mà không phụ thuộc vào Internet.

Tài liệu tham khảo

[1] Espressif Systems. (2023). *ESP32-S3 Technical Reference Manual*. Espressif Systems.

[2] Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel*. Real Time Engineers Ltd.

[3] Warden, P., & Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.